Spatial partitioning on a masked GPU is bistable, and a scheduler that measures can ride it

Draft

Claims traced to `docs/claims-and-evidence.md`

Abstract

We partition a single AMD GPU between concurrent diffusion tenants using CU masks and a step-granular scheduler, and we report three things the hardware does that a static cost model cannot express.

First, the co-run penalty for two same-model tenants on disjoint 16+16 masks is *bistable*: it is either 1.297× or 1.949× the solo step time, the state is drawn once per process, never changes within one, and is identifiable from a single step. Over 160 processes the slow state occurs in 16.9%, [11.4%, 23.6%], and the rate is not stable between campaigns. No predictor of the draw has survived; five have been proposed and retracted.

Second, partitioning *mismatched* tenants is almost free. SDXL beside CogVideoX-2b costs 1.00–1.06 per side at every split, and both tenants advance faster than under an equal share of the whole die: at 4+28, +71.4% and +1.4%; at 28+4, +3.6% and +30.9%. The split decides how the gain divides, not whether there is one.

Third, a dual-ledger scheduler that charges from measurement rather than prediction reduces urgent SLO miss rate by about 10% under same-model co-location, in *both* co-run states, by cutting expensive serial fallbacks by 61–67%. It does not detect the bistable state, which is what it was designed to do; it makes a cheap correction before an expensive one is forced.

We also report a pre-registered negative. On the mismatched workload the grid was designed around — 405 cells, nine policies — the method is 0.56% *worse* than the strongest baseline against a 20% target, because mismatched tenants do not contend and a contention manager has nothing to manage.

Finally, we describe a measurement error worth more than any of the above: a deferred-event instrument produced a two-episode transient with a quantitative account fitting five splits to within 3–9% over a 3× range, and the account was fitting a stale variable. What caught it was running two instruments over the same steps in the same process, where one reported ten times the other inside an interval that physically contained it.

1 Introduction

Serving a latency-sensitive image request beside a long-running video generation on one GPU is a scheduling problem with an unusual property: the two tenants can be given *disjoint sets of compute units* rather than alternating slices of time. On AMD hardware this is `hipExtStreamCreateWithCUMask`; on NVIDIA it is the SM mask. Spatial partitioning is attractive because neither tenant has to be suspended, and a suspended diffusion step is not free to resume.

The premise of any such scheduler is a cost model: how much does a step cost at q units, and how much more does it cost when a peer holds the other $32 - q$? This paper is largely about how badly that second question behaves, and what a scheduler has to do about it.

Contributions.

(1) A measurement of the co-run penalty on gfx1201 showing it is bistable and latched per process (§3).
(2) A measurement showing mismatched tenants partition almost for free, with both sides gaining over rotation at every split (§4).
(3) A dual-ledger scheduler whose probe is worth ≈ 10% where tenants contend, with the mechanism identified as fallback reduction rather than state detection (§5).
(4) A pre-registered negative on the workload the grid was designed around (§6).
(5) A methodological account of an instrument that produced a beautiful and wrong quantitative model (§7).

2 Setup

One Radeon AI PRO R9700 (gfx1201, 32 maskable units, 34.2 GB). SDXL at 768 × 768; CogVideoX-2b at 480 × 720, 9 frames. A step-granular executor exposes `prepare`, `run_step`, `suspend`, `resume` and `finalize`; suspension captures the latent, the step index, the scheduler's internal cursor and the generator state. Interrupting a request between every step and re-granting it at a different mask width produces a latent identical, byte for byte, to the uninterrupted baseline's.

Masks are installed per stream and read back every time. A stream that quietly carried the whole die would produce an unusually *low* externality, which reads as good news, so the readback is evidence rather than a sanity check.

Measuring a step. Step cost is device time between two CUDA events bracketing the step on its own stream. Reading those events with a per-step `synchronize` drains the pipeline and charges the drain to the measurement: it made a 0.1155 s step read as 0.256 s. Reading them one step late avoids the drain but introduces a failure discussed in §7.

3 The co-run penalty is bistable

Two SDXL tenants on disjoint 16+16 masks pay one of two costs.

Three properties make this actionable rather than merely annoying.

It latches. No process was observed in both states: 0 of 8 processes changed across nine later episodes each, although every episode built fresh adapters and re-acquired its streams. Re-forming a pairing does not redraw.

state	processes	mean	range
fast	23 of 30	1.297	1.287–1.303
slow	7 of 30	1.949	1.923–1.962

Table 1: Co-run cost as a multiple of the solo step, one campaign. The clusters do not overlap.

split (sdxl+cog)	SDXL ext	Cog ext	SDXL vs rot.	Cog vs rot.
4+28	1.025	1.063	+71.4%	+1.4%
8+24	1.029	1.052	+65.3%	+8.5%
16+16	1.016	1.047	+42.2%	+22.8%
24+8	1.004	1.006	+17.7%	+29.5%
28+4	1.002	1.010	+3.6%	+30.9%

Table 2: Five processes per split, three episodes, solo and co-run both by device span; 15 measurements per entry, every range within 0.005. Rotation gives a tenant the whole die for its share of the wall clock.

It is per mask pair. Interposing an 8+24 pairing never disturbed the 16+16 state (16 of 16 checks), and 8+24 carried its own independent draw.

One step identifies it. Comparing each episode's first step against its own median gives a mean absolute error of 0.00% over 72 episodes; the per-step series inside an episode is constant.

Pooled over 160 processes the slow state occurs in 27, 16.9%, Clopper–Pearson [11.4%, 23.6%]. The rate is *not* stable across campaigns — 22.7%, 35.0%, 8.3%, 11.7%, with Fisher $p = 0.025$ between the extremes — and we report no predictor. Five were proposed and retracted: working set, power cap, uncontrollable bistability, launch stagger, and prior use of a full-die mask.

The consequence for a cost model is direct. Against rotation at 2×108.9 ms, the fast state makes partitioning worth +11.5% and the slow state worth −25.5%. A model with one externality per width pair is calibrating against a draw it cannot see.

4 Mismatched tenants partition almost for free

Both tenants gain at every split, so partitioning Pareto-dominates rotation here. Neither is paying for the other's gain — which the same-model case never manages — and no draw is observed: the two arrangements behave differently and the die keys its state on the mask pair.

5 A scheduler that measures

The runtime charges each tenant from the *measured* cost of the step it just ran, never from the cost model. Charging from prediction would make the accounting exact by construction and blind: the point of keeping both ledgers is that they can disagree, and the disagreement is what admission control reads.

segment	fast (n=33)	slow (n=10)
deadline actions	+0.62%	−0.57%
probe	**−9.75%**	**−10.87%**
serial fallbacks	−61%	−67%
video goodput	+0.04%	−0.01%

Table 3: Probe intervals [-0.0292, -0.0112] and [-0.0635, -0.0083]; both deadline intervals span zero.

The policy pairs tenants by matched step cost, and probes: if a formed pairing's observed step exceeds its paired expectation by a tolerance, the pairing is dropped for a round. The runtime adds a safety envelope that falls back to a serial action when a pairing has no measured profile or when drift exceeds 15%.

We decompose the gain with three arms from one code path — pairing only, plus the deadline actions, plus the probe — paired by seed, with every process's drawn state recorded.

The probe carries the entire effect and the deadline actions carry none. More interesting is that the two states agree. Had the probe been the slow-state detector it was designed as, its benefit would concentrate in the slow state and vanish in the fast one. Instead it does the same thing in both: it drops a degrading pairing itself, before the envelope has to bail the round out with the whole die — a cheap early correction standing in for an expensive late one.

Robustness. Injecting predictor error into the belief the policy reads, and nowhere else, gives zero safety failures at ±20% across 50 cells, where a safety failure is defined in code as over-committing the die, charging from the cost model when a measurement existed, or charging a measurement taken at another quota. Under +20% over-prediction the prediction-only policy degrades 23% relative and the measuring one 4.6%.

6 A pre-registered negative

On the mismatched workload — 405 cells, nine policies, five seeds, loads $\{0.6, 0.85, 1.05\}$ and bursts $\{2, 4, 8\}$ — the method is 0.56% *worse* than the strongest non-oracle baseline, [+0.0000, +0.0056] over 45 paired configurations, against a target of a 20% improvement.

It follows from §4. Mismatched pairings cost 1.02–1.06, so there is nothing for a contention manager to manage, while a spurious probe still costs a good pairing. The pairing itself does win: FCFS by 6.5%, static partitioning and EDF by about 3%, and at overload FCFS by 17%.

We state this as a negative because it was registered as one. The comparison target was fixed by rule — best baseline mean, computed not chosen — because the same method beats FCFS by 30%, the strongest baseline by 22%, and its own ablation by 4.5% on a single cell, and which opponent a

headline uses is the easiest thing to lose between a table and a sentence.

7 The instrument that fitted an artifact

Reading a step's events one step late avoids draining the pipeline, but the read only lands once the previous step's events have retired. When the measured side's steps are short it never lands, and the variable holds one stale value for an entire episode.

This produced a two-episode "serialisation transient" in which each side appeared to cost the sum of the two solo steps. The account fitted five splits to within 3–9% over a 3× range of that sum. It was a real fit to a stale variable.

What caught it was not reasoning. Running a second instrument — events bracketing the same step on the same stream, which physically *contain* the first — over the same steps in the same process gave 1.022 where the first gave 10.888. Containment made the contradiction unarguable.

We record the working rules this produced, because the same class of error — a property of the measurement presenting itself as a property of the thing measured — occurred nine times: measure the null before varying parameters; state the discriminating criterion and its direction before running; a single co-run measurement is not evidence; the measurement architecture must match the claimed deployment architecture; and make the nuisance variable identical across arms rather than trusting it to balance — seed with position, deadline with arm, and co-run state with arm each confounded a campaign before that was routine.

8 Threats to validity

Single SKU. Every result is from one mask implementation on one card. Nothing here rules out that they depend on that implementation's queueing and arbitration, and the bistability of §3 makes that a live question rather than a formality. No amount of additional cells on the same card substitutes for cross-vendor agreement.

Same-model co-location is not the primary workload. The probe results of §5 are measured there because the mechanism is inert on the mismatched one.

Small n on the slow state. Ten processes. The interval excludes zero; it is not tight.

An unexplained ablation. Making the runtime blind to the externality term *reduced* serial fallbacks where the mechanism and a unit test both say it should increase them. Under investigation.

9 Conclusion

The claim this work set out to make — that spatial partitioning raises utilisation by a fixed factor — is false as stated, and the reason is interesting: the factor is not fixed. It is bistable, latched per process, and depends on whether the co-located tenants contend at all. A scheduler that measures its own steps rides that; one calibrated to a constant takes the draw.